

Problem 1

I recorded a video of the Reading Girl statue beside CSIE Building and extracted frames at a rate of 2 frames per second. I first fed raw images to COLMAP for Structure from Motion (SfM) and generated the mesh using COLMAP's built-in Poisson Surface Reconstruction algorithm. However, the resulting mesh contained many background elements that were not of interest. To address this, I try manually cropping the statue, apply U-2-Net [1] to remove background automatically and a hybrid approach. The results from all four settings are shown in **Figure 1** and **Figure 2** and sample images in **Appendix A**.

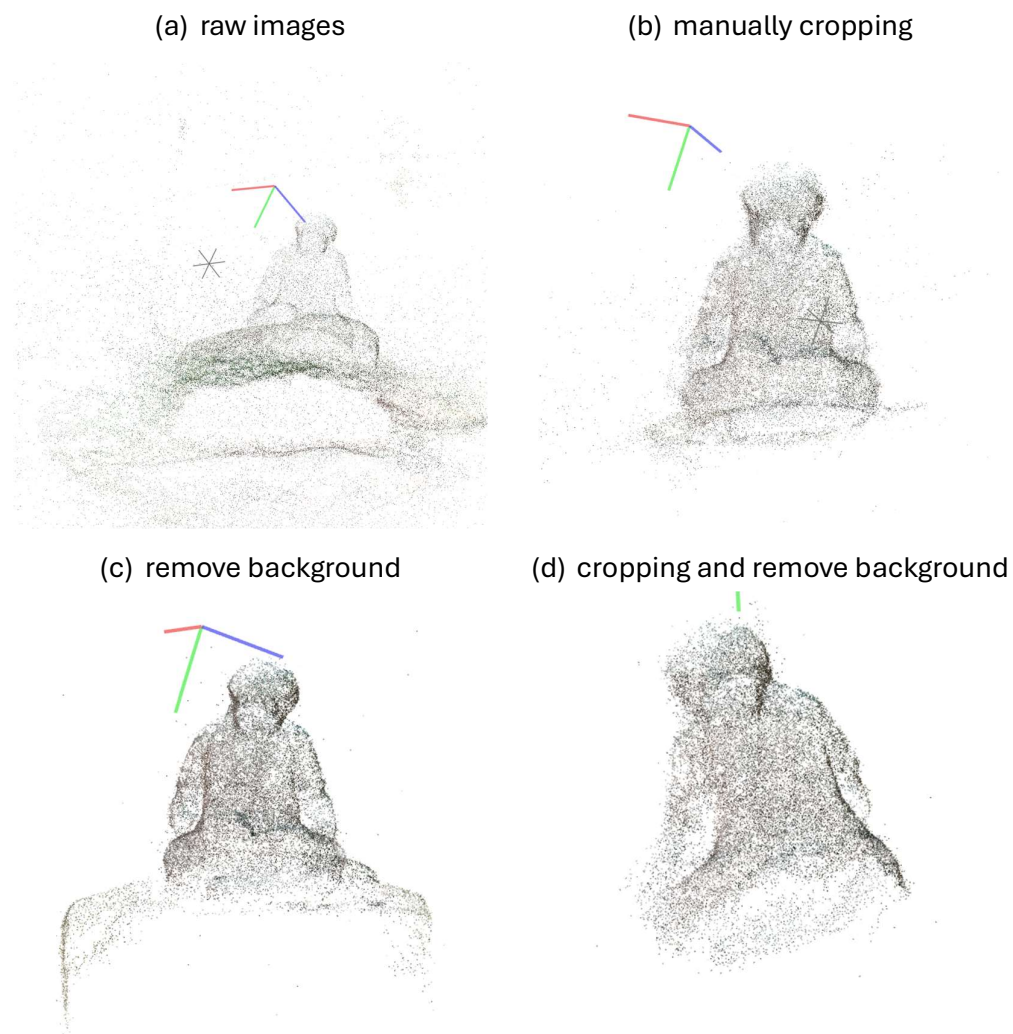


Figure 1. Sparse SfM point clouds with four different settings



Figure 2. Poisson mesh with four different settings

The point clouds appear similar across settings, except that the raw images contain a significant amount of noise. However, the difference is obvious after converting point clouds to meshes: (a) the mesh generated from raw images contains numerous background objects, making it difficult to identify the statue in the center; (b) manual cropping effectively reduces background clutter and produces satisfactory results; (c) and (d) exhibit surface peeling, likely due to U-2-Net removing too many pixels.

Problem 2-1

For each 2D point, I used K-Nearest Neighbor (KNN) search in the 3D descriptor space and applied Lowe's ratio test: if the smallest distance is less than 0.75 times the second smallest distance, the nearest neighbor is considered a good match. This ensures that if the closest and second-closest matches are too similar, the correspondence is likely ambiguous or incorrect and should be discarded. Camera poses were then estimated using only the good matches.

I implemented RANSAC by myself but not P3P, and compared the results with `cv2.solvePnP`, as shown in **Table 1**. My implementation achieved slightly smaller error and improved with iterations.

Table 1. Median of translation errors and rotation errors with different iterations

	Iterations	cv2.solvePnP	My implement
Translation Error Median	100	0.000128929	0.000126453
	1000	0.000128929	0.000119273
Rotation Error Median (rad)	100	4.037169541e-05	3.955854778e-05
	1000	4.037169541e-05	3.851596069e-05

RANSAC pseudo code

Input:

kp_3D: (N, 3)

kp_2D: (N, 2)

cameraMatrix: camera intrinsic matrix

distCoeffs: Distortion Parameters

iterations: number of iterations

reproj_error: (pixels) < reproj_error is viewed as inlier

min_points: minimum number of points in each iteration (≥ 6)

Output:

success: bool, solution found or not

best_rvec, best_tvec: best rotation vector and translation vector

best_inliers: index of best inlier

Algorithm:

for i = 1 to iterations:

 Randomly select *min_points* corresponding (3D, 2D) points

 Attempt to solve *rvec* and *tvec* using PnP (e.g., cv2.solvePnP)

 Use the resulting *rvec* and *tvec* to project all 3D points to 2D

 Calculate the reprojection error

 Find inliers whose reprojection error < *reproj_error*

 If there are more inliers than the current *best_inliers*:

 Update *best_inliers*, *best_rvec* and *best_tvec*

if len(*best_inliers*) >= *min_points*:

 solve *rvec* and *tvec* again using all inliers

else:

 return False, None, None, []

To draw a quadrangular pyramid, five points must be determined, four base points and the apex. I set the base points to be the four corner of photo in camera coordinate system (CCS), if the photo size is *width* × *height*, the corner coordinates in homogeneous are:

$$\left[\left(-\frac{width}{2}, -\frac{height}{2}, 1 \right), \left(-\frac{width}{2}, \frac{height}{2}, 1 \right), \left(\frac{width}{2}, -\frac{height}{2}, 1 \right), \left(\frac{width}{2}, \frac{height}{2}, 1 \right) \right]$$

Then the world coordinates can be solved by inversing the Pinhole camera model:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \approx K \begin{bmatrix} R & t \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = R^{-1} \left(K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} - t \right) = R^T \left(K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} - t \right)$$

K is the camera intrinsic matrix.

The coordinate of optical center in CCS is (0, 0, 0), then the world coordinate is $-R^T t$, connect optical centers to form the trajectory.

Finally, identification the length from optical center to four base points to ensure that the normal of base is the bearing (orientation) of the camera, see

Figure 3.



Figure 3. trajectory and camera poses along with 3d point cloud model

Problem 2-2

Given cube vertices, use bilinear interpolation to get the 3D world coordinate of each point and project it to camera coordinate. Each voxel should be drawn from the furthest to the closest, calculate optical center in world coordinate using $-R^T t$ as mentioned above, then the depth is the distance from each 3D point to optical center. **Figure 3** shows some samples that place a virtual cube successfully, which **Figure 4** shows some samples with strange points appear, which I can't figure out the reason, I try `cv2.projectPoints` and implement a `projectPoints` function, both lead to the same results.

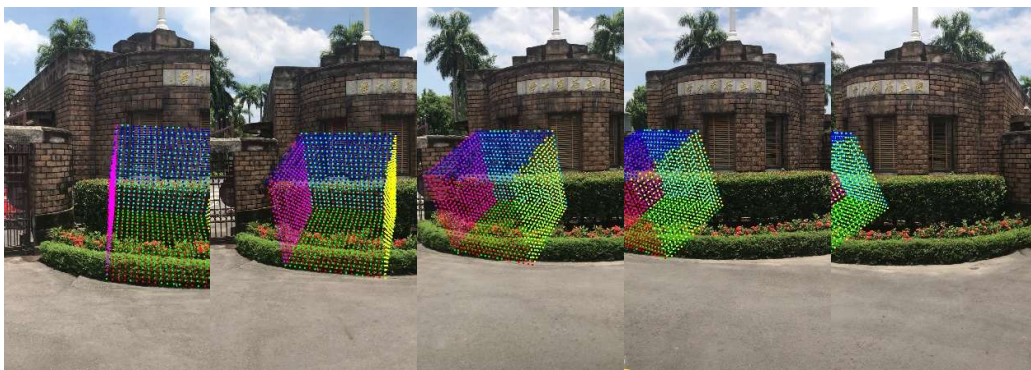


Figure 3. samples that successfully place a virtual cube



Figure 4. Samples with strange points appear

How to execute my codes

Create environment and install requirements

```
conda create -n 3dcv_hw2 python=3.10 -y
```

```
conda activate 3dcv_hw2
```

```
pip install scipy opencv-python open3d tqdm pandas
```

To estimate camera poses, run

```
python 2d3dmatching.py
```

this program will create `est_camera_poses.pkl`

```
python transform_cube.py
```

this program will create `cube_vertices.npy`

After the two files are generated, we can draw cube

```
python draw_cube.py
```

YouTube link

Problem 1: <https://www.youtube.com/watch?v=-78XDkDXtis>

Problem 2: <https://www.youtube.com/watch?v=OpknrYyQQ4I>

Reference

[1] Qin, Xuebin, et al. "U2-Net: Going deeper with nested U-structure for salient object detection." *Pattern recognition* 106 (2020): 107404.

Appendix

A. Sample images with four different settings

(a) raw images



(a) manually cropping



(c) remove background



(d) cropping and remove background

